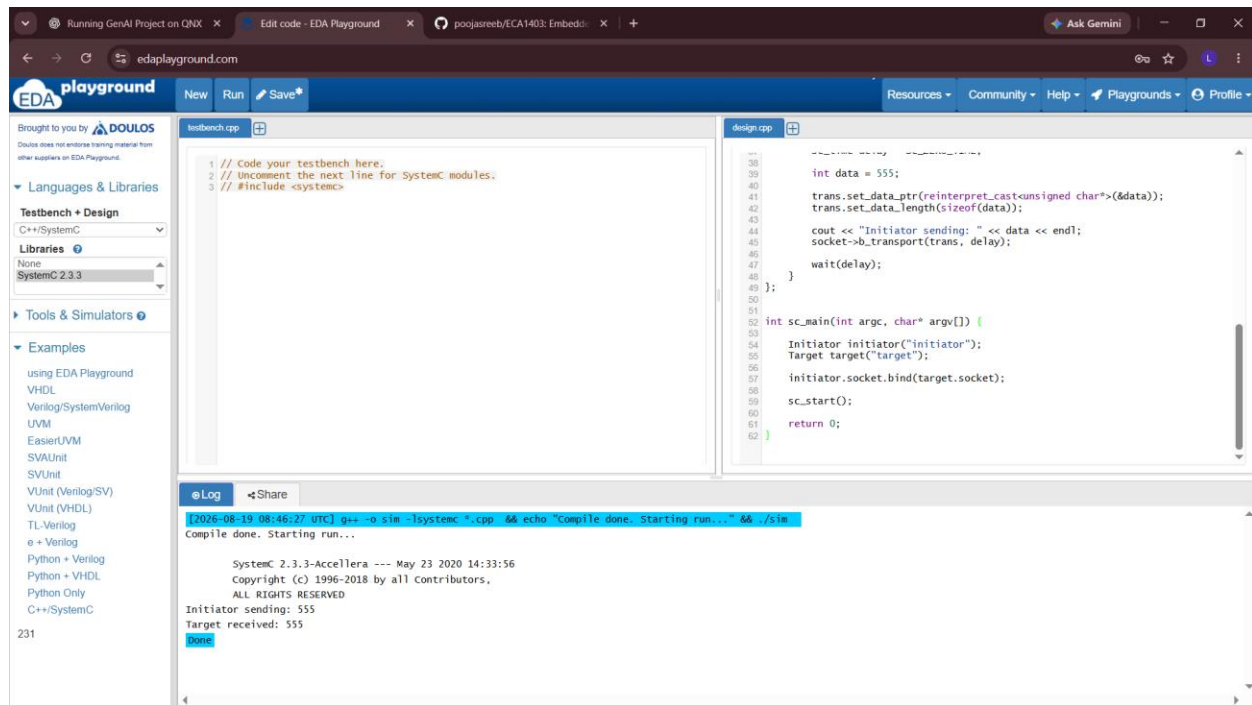




SOURCE CODE

```
#include <systemc>
#include <tlm>
#include <tlm_utils/simple_initiator_socket.h>
#include <tlm_utils/simple_target_socket.h>
```

```
using namespace sc_core;
using namespace tlm;
using namespace std;
```

```
// Target
struct Target : sc_module {
    tlm_utils::simple_target_socket<Target> socket;

    SC_CTOR(Target) : socket("socket") {
        socket.register_b_transport(this, &Target::b_transport);
    }
```

```
void b_transport(tlm_generic_payload& trans, sc_time& delay) {
    int* data = (int*) trans.get_data_ptr();
    cout << "Target received: " << *data << endl;

    delay += sc_time(10, SC_NS);
}
};
```

```

// Initiator
struct Initiator : sc_module {
    tlm_utils::simple_initiator_socket<Initiator> socket;

    SC_CTOR(Initiator) : socket("socket") {
        SC_THREAD(thread_process);
    }

    void thread_process() {
        tlm_generic_payload trans;
        sc_time delay = SC_ZERO_TIME;

        int data = 555;

        trans.set_data_ptr(reinterpret_cast<unsigned char*>(&data));
        trans.set_data_length(sizeof(data));

        cout << "Initiator sending: " << data << endl;
        socket->b_transport(trans, delay);

        wait(delay);
    }
};

```

```

int sc_main(int argc, char* argv[]) {

    Initiator initiator("initiator");
    Target target("target");

    initiator.socket.bind(target.socket);

    sc_start();

    return 0;
}

```